

✓ Intelligent Systems - Assignment 4 - Machine Learning

- Member Names: Javier Gomez & Adei Laka
- Assignment: a04 Machine Learning
- Time spent in the assignment: 10h
- Proportion of effort done by members of the group: 50/50
- Doubts and difficulties that arose during the realization: What kind of plots to use to represent the data. We asked chatgpt and trusted it's decision seeing that the values when seeing the final result of the graph made sense. We also had a lot of trouble with the data of loan approval. We wasted a lot of time until we realised there was a white space before every column name.

General instructions

1. If you use a function of a certain library, do not forget to include the corresponding command to install or load the library
2. Do not forget to comment on the code, especially those non-trivial commands (part of the rating depends on the cleaning of the code)
3. It is strongly recommended to test any implemented function in order to check for its proper operation

Submission

- Print the notebook in a PDF file with all the sections expanded and all the cells executed, in a way all the code can be seen, and all the results are shown.
- Save the notebook as ipynb file.
- Submit both files into ALUD (not in a zip file)

Evaluation

- Total Evaluation - 20%.
 - Correct and error-free execution
 - Efficiency and cleanness of the code
 - Documentation of the code and analysis of results

```
# PLEASE, INCLUDE HERE IF YOU USE ANY ADDITIONAL LIBRARY
packages = c("glue", "ggplot2", "gridExtra", "tidyverse", "caret", "rpart", "e1071", "rpart.plot")
newpack = packages[!(packages %in% installed.packages()[, "Package"])]
if(length(newpack)) install.packages(newpack)
```

```
a=lapply(packages, library, character.only=TRUE)
rm(list = ls())
```

Linear Regression (Your solution here)

✓ Getting the data ready

```
prepare.data = function(file){
  #Get all the information from the CSV
  data = read.csv(file, header = TRUE)
  #Remove ID column
  data = data[ , -1]

  # Function to get the mode (most common value)
  get_mode <- function(v) {
    uniqv = unique(v[!is.na(v)])
    uniqv[which.max(tabulate(match(v, uniqv)))]
  }

  # Replace NA or blanks with the mode for each column
  for (col in names(data)) {
    if (any(is.na(data[[col]]) | data[[col]] == "")) {
      mode_value = get_mode(data[[col]])
      data[[col]][is.na(data[[col]]) | data[[col]] == ""] <- mode_value
    }
  }
  return(data)
}
```

✓ LM Function

```
advertising.lm = function (data){
  # Split data: 75% training
  perc = 0.75
  index.train = createDataPartition(y = data$Sales, p = perc, list = FALSE)

  data.train = data[ index.train, ]
  data.test = data[-index.train, ]

  print(glue(""))
  print(glue("Size of the training set: {nrow(data.train)}"))
  print(glue("Size of the test set: {nrow(data.test)}"))

  # Create Linear Model using training data
```

```

model = lm(Sales ~ ., data = data.train)
print(model)

# Make the prediction using the model and test data
prediction = predict(model, data.test)

# Calculate Mean Average Error
mean_abs_error = mean(abs(prediction - data.test$Sales))
print(glue("Mean Absolute Error: {mean_abs_error}"))
# Calculate Mean Squared Error
mean_squared_error = mean((prediction - data.test$Sales)^2)
print(glue("Mean Squared Error: {mean_squared_error}"))

# Calculate Mean Absolute Percentage Error
mean_abs_percentage_error = mean(abs((prediction - data.test$Sales) / data.test$Sales))
print(glue("Mean Absolute Percentage Error: {mean_abs_percentage_error}%"))

#Return all the values and the model
result = list(
  mean_abs_error = mean_abs_error,
  mean_squared_error = mean_squared_error,
  mean_abs_percentage_error = mean_abs_percentage_error,
  model = model,
  prediction = prediction,
  data.test = data.test)
return(result)
}

```

✓ Executing LM 10 times

```

data = prepare.data("Advertising.csv")

# Variables to store the values the functions will return
mean_abs_error = list()
mean_squared_error = list()
mean_abs_percentage_error = list()
models = list()
predictions = list()
testdata = list()

for (i in 1:10) {
  print(glue("----- Loop {i} -----"))
  result = advertising.lm(data)
  mean_abs_error[[i]] = result$mean_abs_error
  mean_squared_error[[i]] = result$mean_squared_error
  mean_abs_percentage_error[[i]] = result$mean_abs_percentage_error
  models[[i]] = result$model
  predictions[[i]] = result$prediction
  . . . . .
}

```

```

testdata[[i]] = result$data.test
print(glue(""))
}

print(glue("----- Averages -----"))
# Get averages
avg_mean_abs_error = mean(unlist(mean_abs_error))
avg_mean_squared_error = mean(unlist(mean_squared_error))
avg_mean_abs_percentage_error = mean(unlist(mean_abs_percentage_error))
# Print averages
print(glue("Average MAE: {avg_mean_abs_error}"))
print(glue("Average MSE: {avg_mean_squared_error}"))
print(glue("Average MAPE: {avg_mean_abs_percentage_error}%"))

print(glue("----- Standar deviation -----"))
# Get standard deviations
sd_mean_abs_error = sd(unlist(mean_abs_error))
sd_mean_squared_error = sd(unlist(mean_squared_error))
sd_mean_abs_percentage_error = sd(unlist(mean_abs_percentage_error))
# Print standard deviations
print(glue("Standard Deviation MAE: {sd_mean_abs_error}"))
print(glue("Standard Deviation MSE: {sd_mean_squared_error}"))
print(glue("Standard Deviation MAPE: {sd_mean_abs_percentage_error}%"))

----- Loop 1 -----

Size of the training set: 151
Size of the test set: 49

Call:
lm(formula = Sales ~ ., data = data.train)

Coefficients:
(Intercept)          TV          Radio    Newspaper
   3.026230    0.045257    0.197720   -0.005163

Mean Absolute Error: 1.39277062075305
Mean Squared Error: 4.58017486973469
Mean Absolute Percentage Error: 25.0063161521774%

----- Loop 2 -----

Size of the training set: 151
Size of the test set: 49

Call:
lm(formula = Sales ~ ., data = data.train)

Coefficients:
(Intercept)          TV          Radio    Newspaper
   3.138548    0.044734    0.197087   -0.009607

Mean Absolute Error: 1.2074626634718
Mean Squared Error: 3.40022700406100

```

```

mean Squared Error: 3.49833792406198
Mean Absolute Percentage Error: 20.8737857189547%

```

```

----- Loop 3 -----

```

```

Size of the training set: 151
Size of the test set: 49

```

```

Call:
lm(formula = Sales ~ ., data = data.train)

```

```

Coefficients:
(Intercept)          TV          Radio  Newspaper
    2.559900     0.046518     0.194653    -0.001418

```

```

Mean Absolute Error: 1.11382371495855
Mean Squared Error: 1.9027324476918
Mean Absolute Percentage Error: 10.5074956342454%

```

```

----- Loop 4 -----

```

```

Size of the training set: 151
Size of the test set: 49

```

```

Call:
lm(formula = Sales ~ ., data = data.train)

```

```

Coefficients:
(Intercept)          TV          Radio  Newspaper

```

✓ Plotting the last model

```

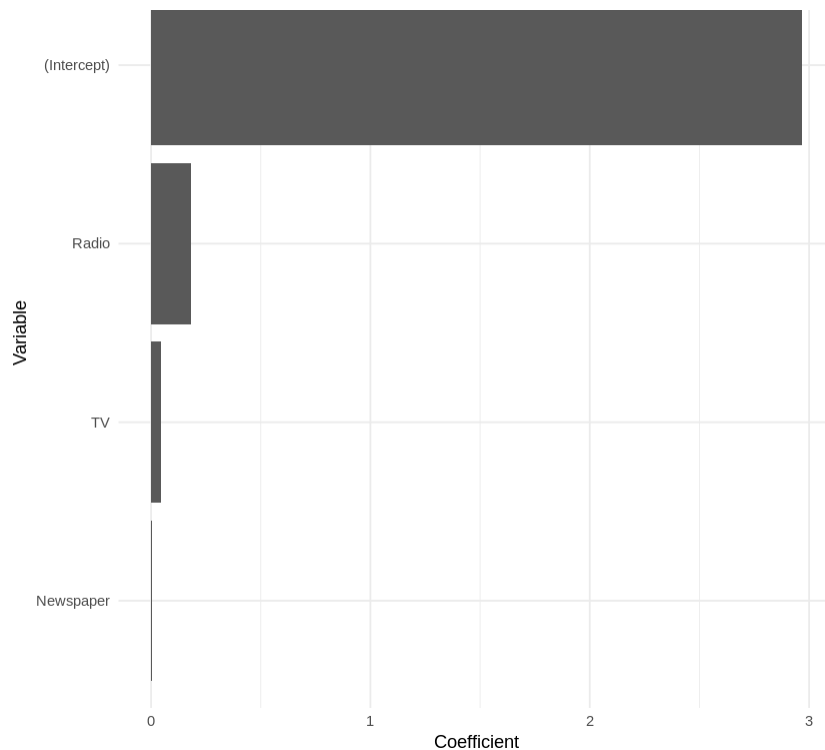
# Get coefficients of last model (iteration 10)
coeffs = coefficients(models[[10]])
coeff_df = data.frame(
  term = names(coeffs),
  estimate = as.numeric(coeffs),
  stringsAsFactors = FALSE
)

# Plot
ggplot(coeff_df, aes(x = reorder(term, estimate), y = estimate)) +
  geom_bar(stat = "identity") +
  coord_flip() +
  labs(title = "Model Coefficients (10th Repetition)",
       x = "Variable",
       y = "Coefficient") +
  theme_minimal()

```

Model Coefficients (10th Repetition)





✓ Top 10 most profitable campaigns

```
data$TotalInvestment = data$TV + data$Radio + data$Newspaper
data$Profitability = data$Sales / data$TotalInvestment
```

```
# Sort data by profitability descending and get top 10
top10 = data %>% arrange(desc(Profitability)) %>% head(10)
print("Top 10 most profitable campaigns:")
print(top10)
```

```
[1] "Top 10 most profitable campaigns:"
```

	TV	Radio	Newspaper	Sales	TotalInvestment	Profitability
1	8.6	2.1	1.0	4.8	11.7	0.4102564
2	31.5	24.6	2.2	9.5	58.3	0.1629503
3	8.4	27.2	2.1	5.7	37.7	0.1511936
4	4.1	11.6	5.7	3.2	21.4	0.1495327
5	27.5	1.6	20.7	6.9	49.8	0.1385542
6	38.2	3.7	13.8	7.6	55.7	0.1364452
7	13.1	0.4	25.6	5.3	39.1	0.1355499
8	19.6	20.1	17.0	7.6	56.7	0.1340388
9	25.6	39.0	9.3	9.5	73.9	0.1285521
10	39.5	41.1	5.8	10.8	86.4	0.1250000

✓ Finding the optimal way to distribute the invest

```
# Step size and budget in units of $1,000
```

```

budget_total = 50
min_per_medium = 5
step = 5

# Create grid of all combinations
combinations = expand.grid(
  TV = seq(min_per_medium, budget_total - 2 * min_per_medium, by = step),
  Radio = seq(min_per_medium, budget_total - 2 * min_per_medium, by = step)
)

# Calculate Newspaper from total - TV - Radio
combinations$Newspaper = budget_total - combinations$TV - combinations$Radio

# Remove combinations where Newspaper is < min
combinations = combinations %>% filter(Newspaper >= min_per_medium)

#Get the last model
final.model = models[[10]]

# Predict sales for all combinations
combinations$PredictedSales = predict(final.model, newdata = combinations)

# Find the best combination
best_combo = combinations %>% arrange(desc(PredictedSales)) %>% head(1)

# Show it
print("Optimal budget allocation to maximize predicted Sales:")
print(best_combo)

[1] "Optimal budget allocation to maximize predicted Sales:"
   TV Radio Newspaper PredictedSales
1   5    40         5         10.59369

```

▼ An End-to-End example 🧐

(but please, in your solution do not do like me and use more than one cell to organize the code, when possible 😊)

```

data = read.csv(file="2020 - 2019 world happiness.csv", sep=";", header = TRUE)
# Remove unuseful columns
data = data %>% select(-Country.or.region, -Overall.rank)

knitr::kable(head(data), caption = "A glipmse of the data")

# Get indexes to do partitions
perc = 0.80
index.train = createDataPartition(y = data$Score, p = perc, list = FALSE)

```

```
# Getting partitions
data.train = data[ index.train, ]
data.test  = data[-index.train, ]
print(glue(""))
print(glue("Size of the training set: {nrow(data.train)}"))
print(glue("Size of the test set: {nrow(data.test)}"))

# Create Linear Model using training data. Formula = all the columns except Score
model = lm(formula = Score ~., data = data.train)
# The point stands for "all the other variables"
print(model)

# Make the prediction using the model and test data
prediction = predict(model, data.test)

# Calculate Mean Average Error
mean_abs_error = mean(abs(prediction - data.test$Score))
print(glue("Mean Absolute Error: {mean_abs_error}"))
```

Table: A glimpse of the data

Score	GDP.per.capita	Social.support	Healthy.life.expectancy	Freedom.to.make.life.choices	Generosity	Perceptions.of.corruption
-----:	-----:	-----:	-----:	-----:	-----:	-----:
7.769	1.340	1.587	0.986	0.596	0.153	0.393
7.600	1.383	1.573	0.996	0.592	0.252	0.410
7.554	1.488	1.582	1.028	0.603	0.271	0.341
7.494	1.380	1.624	1.026	0.591	0.354	0.118
7.488	1.396	1.522	0.999	0.557	0.322	0.298
7.480	1.452	1.526	1.052	0.572	0.263	0.343

Size of the training set: 128

Size of the test set: 28

Call:

```
lm(formula = Score ~ ., data = data.train)
```

Coefficients:

(Intercept)	GDP.per.capita
1.9052	0.6260
Social.support	Healthy.life.expectancy
0.9337	1.2310

Haz doble clic (o pulsa Intro) para editar

▼ Decision Tree (Your solution here)

#Write your code here:

- Use as many cells as needed for clear readability of the code

- Use comments or text cell for pertinent explanations on results or non-trivial code

#SetUp

```
prepare.data = function(file){  
  #Get all the information from the CSV  
  data = read.csv(file, header = TRUE)
```

```
  # Trim whitespace from column names  
  names(data) <- trimws(names(data))
```

```
  #Remove ID column  
  data = data[ , -1]
```

```
  # Function to get the mode (most common value)
```

```
  get_mode <- function(v) {  
    uniqv = unique(v[!is.na(v)])  
    uniqv[which.max(tabulate(match(v, uniqv)))]  
  }
```

```
  # Replace NA or blanks with the mode for each column
```

```
  for (col in names(data)) {  
    if (any(is.na(data[[col]]) | data[[col]] == "")) {  
      mode_value = get_mode(data[[col]])  
      data[[col]][is.na(data[[col]]) | data[[col]] == ""] <- mode_value  
    }  
  }  
  return(data)  
}
```

```
DecisionTree <- function(data){
```

```
  # Split data: 75% training  
  perc = 0.75
```

```
  train_index <- createDataPartition(data$loan_status, p = 0.75, list = FALSE)  
  train_data <- data[train_index, ]  
  test_data <- data[-train_index, ]
```

```
#Train model with depht of 5
```

```
  tree_model <- rpart(  
    loan_status ~ .,  
    data = train_data,  
    method = "class"
```

```

        method = "class",
        control = rpart.control(maxdepth = 5),
        model = TRUE
    )
#Make predictions over the test data
predictions <- predict(tree_model, test_data, type = "class")

#Calculate and store Metrics
#Overall accuracy
overall_acc <- mean(predictions == test_data$loan_status)
#Accuracy per each classes
cm <- confusionMatrix(predictions, as.factor(test_data$loan_status))
if (is.matrix(cm$byClass)) {
  acc_per_class <- cm$byClass[, "Balanced Accuracy"]
} else {
  acc_per_class <- cm$byClass["Balanced Accuracy"]
}
#Accuracy considering only "Graduates" vs "Not Graduates"
acc_grad <- if ("education" %in% colnames(test_data)) {
  grad_filter <- test_data$education %in% c(" Graduate", " Not Graduate")
  mean(predictions[grad_filter] == test_data$loan_status[grad_filter])
} else { NA }
#Accuracy considering only "Graduates" vs "Not Graduates"
acc_self_emp <- if ("self_employed" %in% colnames(test_data)) {
  se_filter <- test_data$self_employed %in% c(" Yes", " No")
  mean(predictions[se_filter] == test_data$loan_status[se_filter])
} else { NA }

return(list(
  model = tree_model,
  overall_accuracy = overall_acc,
  graduate_accuracy = acc_grad,
  self_employed_accuracy = acc_self_emp,
  test_data = test_data,
  predictions = predictions
))
}

#Data
data = prepare.data("loan_approval_dataset.csv")

#Repeat 10 times
results_list <- list()
for (i in 1:10) {
  res <- DecisionTree(data)
  results_list[[i]] <- res
}

#Save result in a data frame

```

```

results_df <- do.call(rbind, lapply(results_list, function(r) {
  data.frame(
    overall_accuracy = r$overall_accuracy,
    graduate_accuracy = r$graduate_accuracy,
    self_employed_accuracy = r$self_employed_accuracy
  )
}))

avg_row <- colMeans(results_df, na.rm = TRUE)
results_df <- rbind(results_df, Average = avg_row)

# Print top 5 most important features from last model
importance <- results_list[[10]]$model$variable.importance
top5 <- sort(importance, decreasing = TRUE)[1:5]
cat("\nTop 5 Most Important Features:\n")
print(top5)

print("Results from 10 runs:")
print(results_df[1:10, ])
print("Average:")
print(results_df[11,])

#rpart.plot(results_list[[10]]$model, main = "Decision Tree (Run 10)")

```

Top 5 Most Important Features:

cibil_score	loan_term	loan_amount
1232.56550	115.28455	33.21880
commercial_assets_value	income_annum	
18.60310	16.31919	

[1] "Results from 10 runs:"

	overall_accuracy	graduate_accuracy	self_employed_accuracy
1	0.9615745	0.9615745	0.9615745
2	0.9671978	0.9671978	0.9671978
3	0.9662605	0.9662605	0.9662605
4	0.9690722	0.9690722	0.9690722
5	0.9765698	0.9765698	0.9765698
6	0.9681350	0.9681350	0.9681350
7	0.9568885	0.9568885	0.9568885
8	0.9634489	0.9634489	0.9634489
9	0.9540769	0.9540769	0.9540769
10	0.9728210	0.9728210	0.9728210

[1] "Average:"

	overall_accuracy	graduate_accuracy	self_employed_accuracy
11	0.9656045	0.9656045	0.9656045

✓ Plot the last tree obtained and identify (with code), the 5 most relevant columns

```

tree.plot = function (model){
  # Plot the three
  rpart.plot(model$model, type = 2, extra = 106, under = TRUE, faclen = 0)
  # Get variable importance from the last model
  importance <- model$model$variable.importance

  # Sort and get top 5
  top5 <- sort(importance, decreasing = TRUE)[1:5]

  # Print them
  cat("Top 5 most important features:\n")
  print(top5)
}

```

```

print(glue("----- Model 8 -----"))
tree.plot(results_list[[8]])
print(glue("----- Model 9 -----"))
tree.plot(results_list[[9]])
print(glue("----- Model 10 -----"))
tree.plot(results_list[[10]])

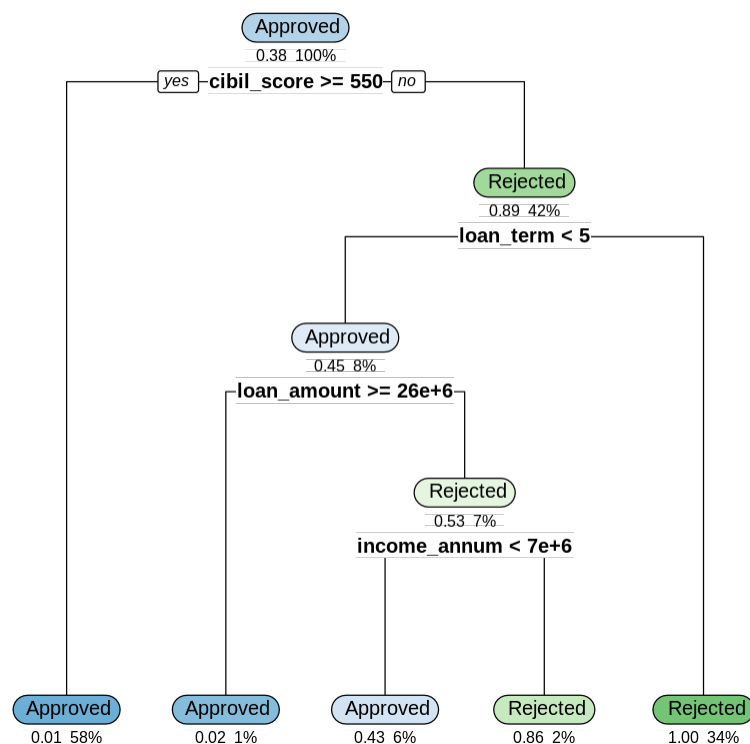
```

----- Model 8 -----

Top 5 most important features:

cibil_score	loan_term	loan_amount
1220.16216	130.33238	23.93523
income_annum	commercial_assets_value	
15.11631	13.51724	

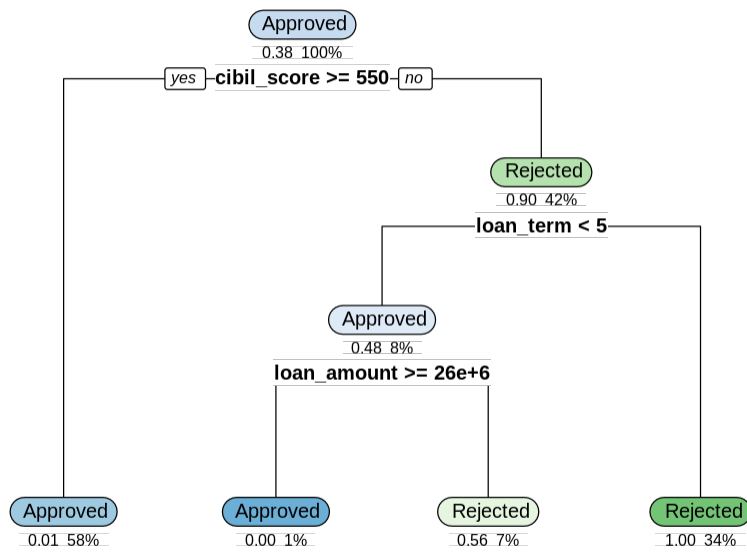
----- Model 9 -----



Top 5 most important features:

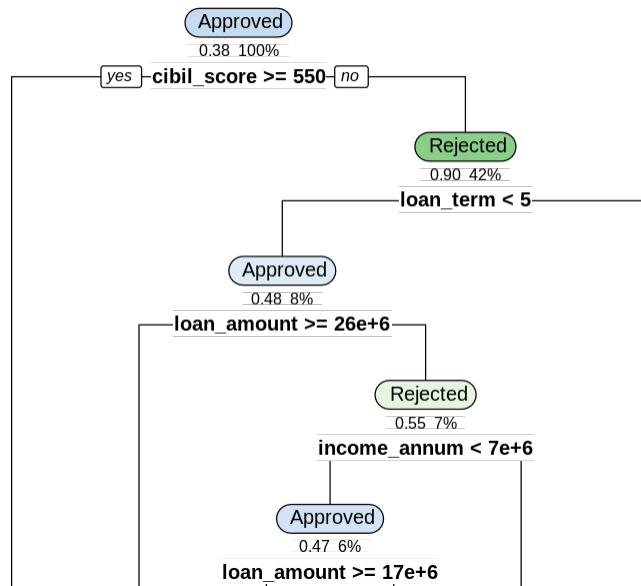
cibil_score	loan_term	loan_amount
1251.029428	110.329438	20.164788
commercial_assets_value	luxury_assets_value	
13.053354	5.085275	

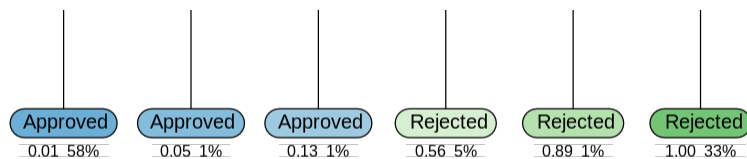
----- Model 10 -----



Top 5 most important features:

cibil_score	loan_term	loan_amount
1232.56550	115.28455	33.21880
commercial_assets_value	income_annum	
18.60310	16.31919	





- ✓ Generate a graphical representation on percentages of actual and predicted values of the model per number of dependents

```
# Get test data and predictions from last run
test_data <- results_list[[10]]$test_data
preds <- results_list[[10]]$predictions

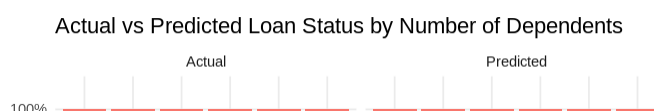
# Make sure column names are trimmed and matched
names(test_data) <- trimws(names(test_data))

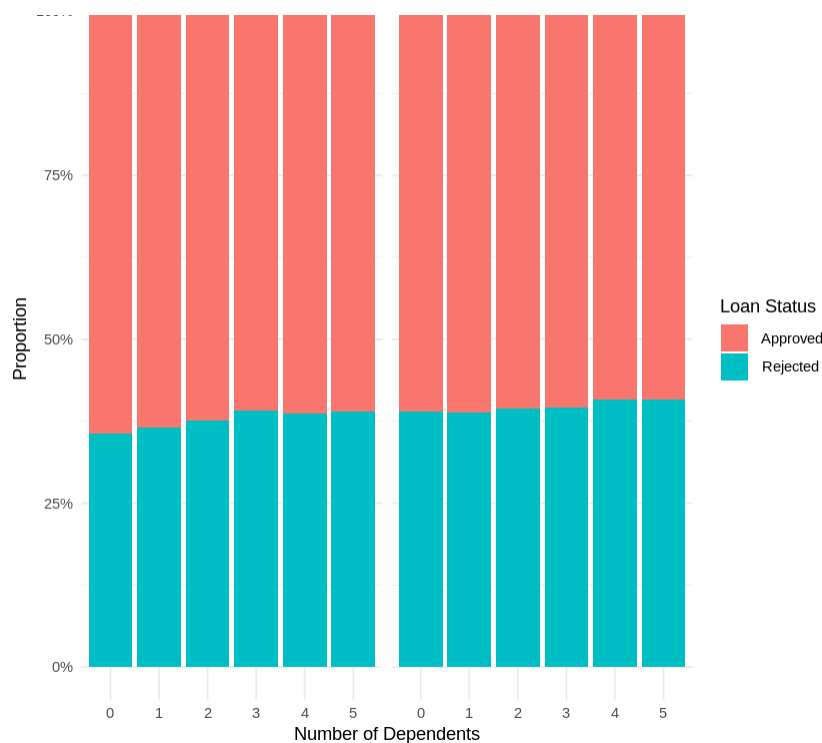
# Combine into a new data frame
compare_df <- data.frame(
  Dependents = test_data$no_of_dependents,
  Actual = as.character(test_data$loan_status),
  Predicted = as.character(preds)
)

# Melt to long format for easier plotting
long_df <- melt(compare_df, id.vars = "Dependents", variable.name = "Type", value.name =

# Calculate proportions
prop_df <- long_df %>%
  group_by(Dependents, Type, Status) %>%
  summarise(Count = n(), .groups = 'drop') %>%
  group_by(Dependents, Type) %>%
  mutate(Percent = 100 * Count / sum(Count))

# Plot
ggplot(prop_df, aes(x = as.factor(Dependents), y = Percent, fill = Status)) +
  geom_bar(stat = "identity", position = "fill") +
  facet_wrap(~Type) +
  labs(title = "Actual vs Predicted Loan Status by Number of Dependents",
       x = "Number of Dependents", y = "Proportion",
       fill = "Loan Status") +
  scale_y_continuous(labels = scales::percent_format()) +
  theme_minimal()
```





Identify (if any) clients with rejected loans such as if would
 ✎ change from Not graduated to Graduated would have had approved loan.

```
# Get last test data and model
test_data <- results_list[[10]]$test_data
model <- results_list[[10]]$model
predictions_orig <- results_list[[10]]$predictions

# Ensure column names are cleaned
names(test_data) <- trimws(names(test_data))

# Get relevant rejected clients who are Not Graduates
rejected_ng <- test_data[test_data$loan_status == "Rejected" & test_data$Education == "No

if (nrow(rejected_ng) > 0) {
  # Simulate as if they were Graduates
  rejected_ng_mod <- rejected_ng
  rejected_ng_mod$Education <- "Graduate"

  # Predict with the modified data
  predicted_if_graduate <- predict(model, rejected_ng_mod, type = "class")

  # Compare: who would change from Rejected to Approved
  changed_clients <- rejected_ng[predicted_if_graduate == "Approved", ]
```

```
# Print them
print("Clients who would be approved if they were graduates:")
print(changed_clients)
} else {
  print("No rejected clients with 'Not Graduate' status found in test data.")
}
```

```
[1] "No rejected clients with 'Not Graduate' status found in test data."
```

✓ For each client with rejected loan, identify the needed income (if any) to get an approval

```
# Get test data and model
test_data <- results_list[[10]]$test_data
model <- results_list[[10]]$model
names(test_data) <- trimws(names(test_data))

# Filter only rejected clients
rejected_clients <- test_data[trimws(test_data$loan_status) == "Rejected", ]

# Initialize result list
income_adjustments <- data.frame()

for (i in 1:nrow(rejected_clients)) {
  client <- rejected_clients[i, ]
  original_income <- as.numeric(client$income_annum)

  if (is.na(original_income)) next

  new_income <- original_income
  approved <- FALSE

  while (!is.na(new_income) && new_income <= 2 * original_income) {
    client$income_annum <- new_income

    # Prepare features for prediction
    features_only <- client
    features_only$loan_status <- NULL

    # Predict
    pred <- predict(model, features_only, type = "class")

    # Check for approval
    if (trimws(as.character(pred)) == "Approved") {
      approved <- TRUE
      income_needed <- new_income
    }
  }
}
```



```

        break
    }else{
        income_needed <- NA
    }

    new_income <- new_income + 100000
}

#income_needed <- if (approved) new_income else NA

income_adjustments <- rbind(income_adjustments, data.frame(
  ClientIndex = rownames(rejected_clients)[i],
  OriginalIncome = original_income,
  RequiredIncome = income_needed
))
}

# Print result
cat("\n Required income adjustments for rejected clients:\n")
print(income_adjustments)

```

 Required income adjustments for rejected clients:

	ClientIndex	OriginalIncome	RequiredIncome
1	6	4800000	NA
2	8	5700000	NA
3	15	1900000	NA
4	32	5700000	NA
5	33	800000	NA
6	40	1500000	NA
7	46	8000000	NA
8	57	4300000	NA
9	63	8800000	NA
10	70	1900000	NA
11	88	8700000	NA
12	110	3300000	NA
13	112	7400000	NA
14	117	5800000	NA
15	129	700000	NA
16	144	6600000	NA
17	150	2000000	NA
18	163	9700000	NA
19	168	1800000	NA
20	169	9100000	NA
21	175	6600000	NA
22	190	7400000	NA
23	204	6900000	NA
24	226	5200000	NA
25	249	8400000	NA
26	251	4200000	NA
27	271	5200000	NA
28	280	3900000	NA
29	282	6900000	NA
30	294	2900000	NA
31	333	3300000	NA

31	320	2300000	NA
32	330	200000	NA
33	346	4200000	NA
34	351	6700000	NA
35	379	5000000	NA
36	396	800000	800000
37	402	3700000	NA
38	409	7500000	NA
39	411	7100000	NA
40	423	5600000	NA
41	431	9300000	NA
42	445	3600000	NA
43	448	8600000	NA
44	450	2900000	NA
45	465	6100000	NA
46	467	500000	NA
47	471	6900000	NA
48	494	300000	NA
49	495	2300000	NA
50	507	4200000	NA
51	513	7800000	NA
52	523	8700000	NA
53	533	6900000	NA
54	557	9700000	NA
55	571	2300000	NA
56	593	800000	NA

For each client with approved loan, identify the

- ✦ maximum loan amount (if any) he could have asked and still have a positive response

```
# Get test data and model
test_data <- results_list[[10]]$test_data
model <- results_list[[10]]$model
names(test_data) <- trimws(names(test_data))

# Filter only approved clients
approved_clients <- test_data[test_data$loan_status == " Approved", ]

# Initialize result data frame
loan_adjustments <- data.frame()

# Loop over each approved client
for (i in 1:nrow(approved_clients)) {
  client <- approved_clients[i, ]
  original_loan <- client$loan_amount

  if (is.na(original_loan)) next
```

```

# Set starting loan and cap
new_loan <- original_loan
max_loan <- original_loan * 2
still_approved <- TRUE

while (new_loan <= max_loan) {
  test_client <- client
  test_client$loan_amount<- new_loan
  test_client$loan_status<- NULL # Remove target for prediction

  prediction <- predict(model, test_client, type = "class")

  if (prediction == " Rejected") {
    still_approved <- FALSE
    break
  }

  new_loan <- new_loan + 100000
}

# Record the result (last approved loan amount)
max_approved_loan <- if (!still_approved) new_loan - 100000 else new_loan

loan_adjustments <- rbind(loan_adjustments, data.frame(
  ClientIndex = rownames(approved_clients)[i],
  OriginalLoanAmount = original_loan,
  MaxApprovedLoanAmount = max_approved_loan
))
}

# Output results
cat("\n📊 Maximum loan amounts while still getting approval (for approved clients):\n")
print(loan_adjustments)

```

📊 Maximum loan amounts while still getting approval (for approved clients):

	ClientIndex	OriginalLoanAmount	MaxApprovedLoanAmount
1	7	33000000	66100000
2	9	2200000	4500000
3	24	1400000	2900000
4	28	5600000	11300000
5	35	11900000	23900000
6	37	6200000	12500000
7	53	7600000	15300000
8	54	23000000	46100000
9	60	18200000	36500000
10	62	30500000	61100000
11	78	33500000	67100000
12	79	29400000	58900000
13	89	8000000	16100000
14	98	25000000	50100000
15	99	29400000	58900000
16	...	...	...

16	102	11000000	23000000
17	120	32900000	65900000
18	123	32900000	65900000
19	125	19500000	39100000
20	127	800000	1700000
21	137	16700000	33500000
22	142	7500000	15100000
23	153	20600000	41300000
24	184	23100000	46300000
25	187	5000000	10100000
26	193	2900000	5900000
27	194	13200000	26500000
28	197	1500000	3100000
29	205	30100000	60300000
30	208	15400000	30900000
31	215	15200000	30500000
32	216	16400000	32900000
33	219	17000000	34100000
34	225	1700000	3500000
35	238	15500000	31100000
36	246	28100000	56300000
37	254	30900000	61900000
38	257	35400000	70900000
39	259	38800000	77700000
40	261	8100000	8000000
41	272	27900000	55900000
42	281	3600000	7300000
43	299	2600000	5300000
44	305	23600000	47300000
45	313	5100000	10300000
46	314	14100000	28300000
47	337	18300000	36700000
48	345	5100000	10300000
49	354	7000000	14100000
50	362	14100000	14000000
51	364	26900000	53900000
52	366	21900000	43900000
53	367	12500000	25100000
54	372	3200000	6500000
55	374	25500000	51100000
56	383	29000000	58100000

▼ An End-to-End example 🧐

(but please, in your solution do not do like me and use more than one cell to organize the code, when possible 😊)

```
data = read.table(file="2020 covid-19-recommendations.tab", header = T, stringsAsFactors = F)
# Remove unuseful columns
knitr::kable(head(data), caption = "A glimpse of the data")

# Get indexes to do partitions
```

```
perc = 0.80
index.train = createDataPartition(y = data$TARGET, p = perc, list = FALSE)

# Getting partitions
data.train = data[ index.train, ]
data.test  = data[-index.train, ]
print(glue(""))
print(glue("Size of the training set: {nrow(data.train)}"))
print(glue("Size of the test set: {nrow(data.test)}"))

# Create Tree Model using training data. Formula = all the columns except Score
model = rpart(formula = TARGET ~., data = data.train)
print(model)

# Make the prediction using the model and test data
prediction = predict(model, data.test, type= "class")

# Display Confusion Matrix
print(glue(""))
mat = caret::confusionMatrix(data.test$TARGET, prediction)
mat$table

# Print the accuracy
print(glue(""))
print(glue("The model hits in {round(100*mat$overall[[1]],2)} % of the patients"))
```

Table: A glimpse of the data

	FEVER	COUGH	BREATHING_DIFFICULTY	FATIGUE	SPUTUM	MUSCLE_PAIN	HEADACHE	SORE_THROAT	SHIVERS	NAUSEAS	NASAL_CONGESTION	DIARRHOEA	BLOOD_EXPECTORATION	CONJUNCTIVITIS	TARGET
0	0	1	0	0	0	0	0	0	0						
0 stay-at-home			0	0											
0	0	0	0	0	0			0						1	
0	0	0	0	0	0			0							
0 stay-at-home			0	1	1			0						0	
0	0	0	1	0				0							
0 go-to-hospital			0	0	0			0						0	
0	1	1	0	0	0			0							
0	1	0	0	0				0							
0 stay-at-home			0	0	0			0						0	
0	0	0	0	0	0			0							
0	0	0	0	0	0			0							
0 stay-at-home			1	0	0			0						0	
0	0	0	0	0				0							
0 stay-at-home			0	0				0							

```
Size of the training set: 6401
Size of the test set: 1599
n= 6401
```

```
node), split, n, loss, yval, (yprob)
      * denotes terminal node
```

```
1) root 6401 1605 stay-at-home (0.25074207 0.74925793)
  2) FEVER>=0.5 2127 698 go-to-hospital (0.67183827 0.32816173)
    4) COUGH>=0.5 1237 130 go-to-hospital (0.89490703 0.10509297) *
    5) COUGH< 0.5 890 322 stay-at-home (0.36179775 0.63820225)
      10) BREATHING_DIFFICULTY>=0.5 239 84 go-to-hospital (0.64853556 0.35146444) *
      11) BREATHING_DIFFICULTY< 0.5 651 167 stay-at-home (0.25652842 0.74347158)
        22) CONJUNCTIVITIS>=0.5 20 0 go-to-hospital (1.00000000 0.00000000) *
        23) CONJUNCTIVITIS< 0.5 631 147 stay-at-home (0.23296355 0.76703645)
          46) DIARRHOEA>=0.5 44 13 go-to-hospital (0.70454545 0.29545455) *
          47) DIARRHOEA< 0.5 587 116 stay-at-home (0.19761499 0.80238501) *
  3) FEVER< 0.5 4274 176 stay-at-home (0.04117922 0.95882078) *
```

Reference

Conclusions

Linear regression

- To our surprise, radio seems to be the most valuable medium to advertise in. We call this a surprise because we don't listen to the radio at all. At first we checked our code again but even after running it multiple times, the result is always the same.
- Seeing the coefficients we got and how they favour radio, it's no surprise that when distributing the money for the most efficient campaign, it goes all in on radio and gives the minimum to the other two.
- Personally, we wouldn't use this prediction model in a real scenario because to us, the data and the results feel outdated. Specially taking into account that there is nothing about social media on it. We guess that this shows the importance of having recent data as well as having as many data points as possible.

Decision tree

- The predictions that we make are similar to the real data.
- The tree decides to check the civil score and the loan term first, and that can cause that people with a high civil score will be able to ask for a loan with a crazy amount of money and can happen in the opposite, a person with a low civil score and a low loan term will never be able to get the loan because it doesn't matter the income of the person

